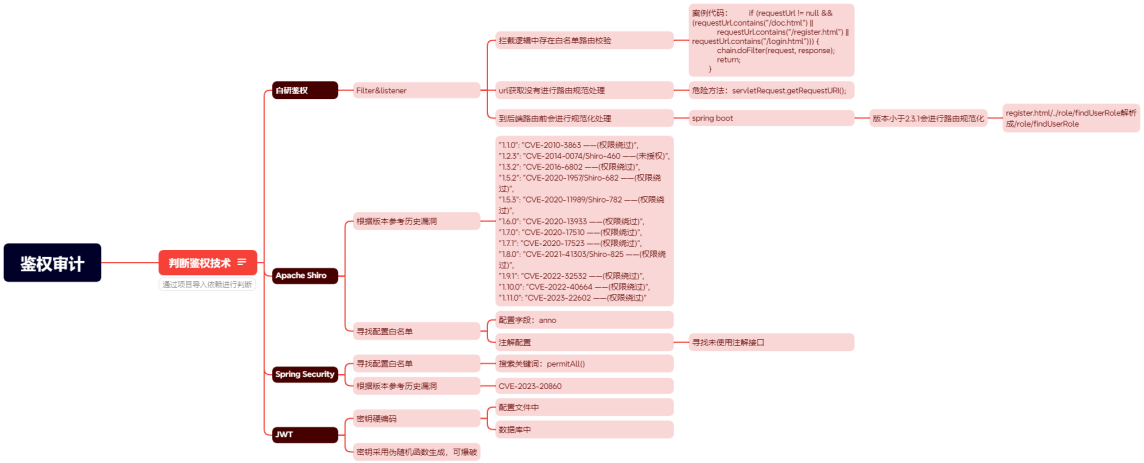
思路图解



拿到一套JAVA项目，关于鉴权方面的挖掘我们要先去判断，项目采用哪些方法进行鉴权

java项目中常见的鉴权方法有

- **自写鉴权**
- **Apache Shiro鉴权**
- **Spring Security鉴权**

JWT只是一种认证方式，不是鉴权框架，通常是搭配这三种鉴权框架使用的，这最后说

我们一个个来

自写鉴权

自写鉴权占据java项目中鉴权问题80%，可以说只要是市面上能够看到的某某源码鉴权绕过后台RCE这种文章项目的鉴权方式多为自写鉴权

我们来浅谈下它的绕过逻辑

一次自写鉴权的绕过，有下面三个条件

1. 项目鉴权中存在白名单匹配
2. 对我们请求路由的获取没有进行**路径规范化**处理
3. 当我们绕过过滤器或者拦截检测后，到后端的servlet或者controller前进行了**路径规范化**

我们一个个来说道

1、项目鉴权中存在白名单匹配

这个多数项目都会有，为什么呢？我们从开发者的角度来思考这个问题

首先，java不同于php，java它的功能点首先采用的路由的形式，简单来说就说路由对应方法，而php是通过路由对应落地文件的形式，在php中我们可以通过划分文件夹来区分前台功能点与后台，但是java的特性来说，我们是通过定义路由检测来区分的，比如在后台的方法前定义一个主路由，例如：admin，admin开头访问的方法都进行管理员身份鉴权，还有个原因，目前java开发的程序多为管理系统，也就是很多系统是没有前台这个概念的，只有后台，然后有权限划分

在上节课我也介绍了javaweb中鉴权校验方法一般会写在过滤器或者拦截器，那么在没有前台这个情况下，我们要拦截访问路由就不会一个个路由定义对应拦截器，而是直接定义全局也就是@WebFilter(filterName = "LogCostFilter", urlPatterns = {"/*"}),访问所有路由都会提前走鉴权方法，那么这里师傅们应该也想到了，一些静态图片资源呢？还有登录接口呢？我要登录后台肯定要访问登录接口，但是它又是全部拦截，不是连登录接口都不能访问，不能访问我咋登录呢？所以就会出现白名单接口，开发者会在拦截器或者过滤器中加上白名单路由，例如下面代码

```
//获取访问路由
String requestUrl = servletRequest.getRequestURI();
// 检查请求URL是否为特定页面，如果是，则不进行拦截
if (requestUrl != null && (requestUrl.contains("/doc.html") ||
    requestUrl.contains("/register.html") || requestUrl.contains("/login.html")))
{
    // 对于这些特定页面的请求，直接继续执行过滤链，不进行拦截
    chain.doFilter(request, response);
    return;
}
```

并且这里白名单的判断必须得是包含，也就是模糊匹配

那么满足这个条件会就要考虑第二个了

2、对我们请求路由的获取没有进行路径规范化处理

什么是路径规范化呢？就比如当我们访问/user/../admin在浏览器中，它会解析成/admin，这个就说**路径规范化**，也可以说是**路径解析**，我们把上面的代码拿下来

```
//获取访问路由
String requestUrl = servletRequest.getRequestURI();
// 检查请求URL是否为特定页面，如果是，则不进行拦截
if (requestUrl != null && (requestUrl.contains("/doc.html") ||
    requestUrl.contains("/register.html") || requestUrl.contains("/login.html")))
{
    // 对于这些特定页面的请求，直接继续执行过滤链，不进行拦截
    chain.doFilter(request, response);
    return;
}
```

这里其实是可以绕过的，这里关于url的获取使用的是getRequestURI()方法，它是不会对我们访问的url进行路径规范化的，也就说说这里如果后续代码中没有类型..的处理或者检测这里就满足条件二，我们就可以通过构造例如：login.html/../caigo(后台功能路由)，或者/caigo;/login.html来进行尝试绕过

3、当我们绕过过滤器或者拦截检测后，到后端的servlet或者controller前进行了路径规范化

什么意思呢？当我们进行第二步后，其实已经绕过拦截器或过滤器了，那么放行后的路由是login.html/../caigo(后台功能路由)，或者/caigo;/login.html，我们要关注它能不能解析成/caigo，这个就是路径规范化，如果到servlet或者controller的时候没进行路径规范化解析，/caigo;/login.html它肯定是识别不到的对吧，找不到对应路由啊，这个也是个关键点，不然鉴权是绕过了，但是触发不到对应路由也没意义对吧

两种绕过方式我们都说下

/login.html/../caigo

这个我测试tomcat项目，9最新版仍然可以，会解析成/caigo

spring boot项目的路由解析使用的是springframework，如果项目中spring boot版本大于2.3.0，那么无法解析成/caigo

✅ Spring Boot 2.3.0（基于 Spring Framework 5.2.x）
使用 org.springframework.util.AntPathMatcher 或 PathPatternParser 解析 URL。
默认会进行 路径规范化（normalize），包括处理 ..、. 等符号。
所以 /user/../admin 会被简化为 /admin。
🔒 Spring Boot 2.3.1+（基于 Spring Framework 5.3.x 开始）
引入了更严格的路径匹配策略，默认 禁用路径中的 .. 解析，防止路径穿越攻击（Path Traversal）。
这个变更源于 Spring Framework 对安全性的增强，具体参考：
Spring Framework GitHub PR #25174
Spring Framework 5.3 引入了 useCaseSensitiveMatch 和 useTrailingSlashMatch 等配置项，并默认关闭了 .. 的解析。

/caigo;/login.html

Tomcat 的 **URL 解析机制** 会：

1. 将 `;` 视为 **参数分隔符**（类似 `?` 但非标准）。
2. 默认情况下，`/caigo;/login.html` 会被截断，只保留 `;` 之前的部分，即 `/caigo`。
3. 这种行为可能导致安全问题（如 **绕过权限检查**）。

Tomcat 7.x / 8.x / 9.x / 10.x 均默认启用此行为。

主要影响：如果应用的路由鉴权仅检查 `request.getRequestURI()`，攻击者可能通过 `;` 绕过路径匹配规则。

部分get需要传参路由这样绕过

```
/admin;login?username=123
```

Spring boot测试了最新的3.5.0，同理

```
/test;admin?name=123
```

会解析成/test?name=123

所以实战中推荐/caigo;/login.html这种方式尝试绕过

实战案例

华夏erp

```
            chain.doFilter(request, response);
            return;
        }
        if (requestUrl != null && (requestUrl.contains("/doc.html") ||
            requestUrl.contains("/register.html") || requestUrl.contains("/login.html"))) {
            chain.doFilter(request, response);
            return;
        }
```

绕过poc

```
/login.html/../后台路由
```

report

```
        }

        // swagger相关的直接放行
        if (uri.contains("swagger-ui") || uri.contains("swagger-resources")) {
            filterChain.doFilter(request, response);
            return;
        }


        if (SLASH.equals(uri) || SLASH.concat(BusinessConstant.SLASH).equals(uri)) {
            if (BusinessConstant.SLASH.equals(uri)) {
                response.sendRedirect( s: "/index.html");
                return;
```

绕过poc：

```
/后台路由;swagger-ui
```

Apache Shiro鉴权

关注历史绕过漏洞，和白名单

历史漏洞根据项目导入shiro版本进行尝试

```
"1.1.0": "CVE-2010-3863 --(权限绕过)",
"1.2.3": "CVE-2014-0074/Shiro-460 --(未授权)",
"1.3.2": "CVE-2016-6802 --(权限绕过)",
"1.5.2": "CVE-2020-1957/Shiro-682 --(权限绕过)",
"1.5.3": "CVE-2020-11989/Shiro-782 --(权限绕过)",
"1.6.0": "CVE-2020-13933 --(权限绕过)",
"1.7.0": "CVE-2020-17510 --(权限绕过)",
"1.7.1": "CVE-2020-17523 --(权限绕过)",
"1.8.0": "CVE-2021-41303/Shiro-825 --(权限绕过)",
"1.9.1": "CVE-2022-32532 --(权限绕过)",
"1.10.0": "CVE-2022-40664 --(权限绕过)",
"1.11.0": "CVE-2023-22602 --(权限绕过)"
```

白名单有两种，一种是配置文件中配置，还有一种通过注解配置无需鉴权路由

这里说下配置文件的

Filter Name	功能
anno	不需要授权、登录就可以访问。eg:/index
authc	**需要登录授权才能访问。eg：/用户中心**
authcBasic	Basic HTTP身份验证拦截器
logout	退出拦截器。退出成功后，会redirect到设置的/URI
noSessionCreation	不创建会话连接器
perms	授权拦截器:perm['user:create']
port	端口拦截器.eg:port[80]
rest	rest风格拦截器
roles	角色拦截器。eg：role[administrator]
ssl	ssl拦截器。通过https协议才能通过
user	用户拦截器。eg：登录后（authc），第二次没登陆但是有记住我(remmbner)都可以访问。

这个白名单一般不是在项目配置文件就是在拦截器中

注解的话，要看写法，如何判断很简单，找个前台功能路由，在代码中全局搜索就知道是配置文件还是注解了

实战案例

tumo

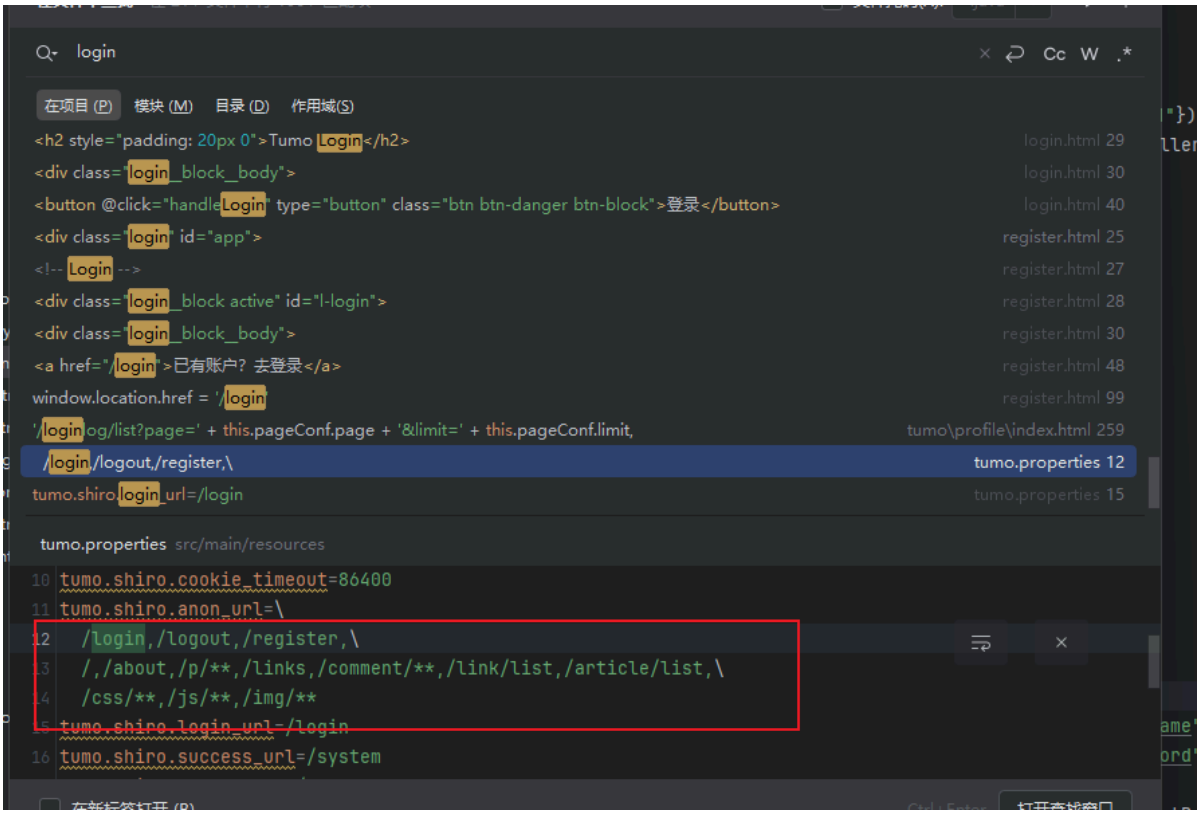
先找个前台未鉴权接口

```
    @Autowired
    private Md5Util md5Util;

    @Autowired
    private LoginLogService loginLogService;

    /**
     * 登录接口
     *
     * @param username
     * @param password
     * @return
     */
    @PostMapping("/login")
    public R login(@RequestParam(value = "username", required = false) String username,
                   @RequestParam(value = "password", required = false) String password) {
        Subject subject = getSubject();
        String encrypt_password = md5Util.encryptPassword(password);
        UsernamePasswordToken token = new UsernamePasswordToken(username, encrypt_password);
        try {
            subject.login(token);
            HttpServletRequest request = HttpContextUtil.getHttpServletRequest();
```

这里没有看到注解，全局搜索对应接口



Spring Security鉴权

关注历史绕过漏洞，和白名单

历史漏洞根据项目导入版本进行尝试

```
cve-2018-1199 Spring Security和Spring Framework认证绕过漏洞
cve-2018-1258 Spring Framework及Spring Security 未授权访问漏洞
CVE-2019-3795 使用Spring Security构建的secureRandom实例时的不安全随机性
CVE-2022-22978 Spring Security RegexRequestMatcher 认证绕过漏洞
CVE-2022-31692 Spring Security forward/include 认证绕过漏洞
CVE-2023-20862 Spring Security Logout实现不当
CVE-2023-34034 Spring Security 路径匹配权限绕过漏洞
CVE-2023-34035 Spring Security鉴权规则错误配置风险
CVE-2024-22234 Spring Security isFullyAuthenticated 权限绕过漏洞
CVE-2024-22257 Spring Security AuthenticatedVoter 权限绕过漏洞
CVE-2024-38810 Spring Security @AuthorizeReturnObject 权限绕过漏洞
CVE-2024-38821 Spring Security 静态资源未授权访问漏洞
CVE-2024-38827 Spring Security 大小写敏感校验鉴权绕过风险
CVE-2025-22228 Spring Security 超长密码处理不当漏洞
CVE-2025-22223 Spring Security @EnableMethodSecurity 注解验证不当漏洞
CVE-2025-22235 Spring Security EndpointRequest鉴权不当漏洞
CVE-2025-41232 Spring Security 私有方法授权绕过漏洞
```

白名单

```
// 方式1：完全忽略（不经过Security过滤器链）
web.ignoring().antMatchers("/css/**", "/js/**");

// 方式2：通过permitAll()放行（经过过滤器链但无需认证）
http.authorizeRequests().antMatchers("/public/**").permitAll();.permitAll()
```

全局搜索permitAll()即可

实战案例

crmeb_java

```
permitAll()                                                    9/12
38    public class WebSecurityConfig extends WebSecurityConfigurerAdapter {
100       protected void configure(HttpSecurity http) throws Exception {
122           .antMatchers("/api/codegen/code").permitAll()
123   //        .antMatchers("/wx/user/*/login","/citylife/nocheck/**").anonymous()
124           .antMatchers(
125                   HttpMethod.GET,
126                   "/*.html",
127                   "/**/*.html",
128                   "/**/*.css",
129                   "/**/*.js"
130           ).permitAll()
131           .antMatchers("/profile/**").anonymous()
132           .antMatchers("/common/download**").anonymous()
133           .antMatchers("/common/download/resource**").anonymous()
134           .antMatchers("/doc.html").permitAll()
135           .antMatchers("/swagger-resources/**").permitAll()
136           .antMatchers("/webjars/**").permitAll()
137           .antMatchers("/v2/**").permitAll()
138           .antMatchers("/swagger-ui.html/**").permitAll()
139           .antMatchers("/*/api-docs").anonymous()
140           .antMatchers("/druid/**").anonymous()
141           .antMatchers("/captcha/get", "/captcha/check").anonymous()
142           .antMatchers("/api/admin/payment/callback/**").anonymous()
143           .antMatchers("/api/public/**").anonymous()
144           // 除上面外的所有请求全部需要鉴权认证
145           .anyRequest().authenticated()
146           .and() HttpSecurity
```

JWT

这个是认证方法，jwt的特点是什么呢？通过密钥加密，并且是可逆的，那么发现项目使用jwt验证用户身份标识时，我们关注两个点

1、密钥是否能获取到

2、jwt数据中的身份标识能否伪造

我们来细说这两部分

1、密钥是否能获取到

也是两种情况

1、硬编码

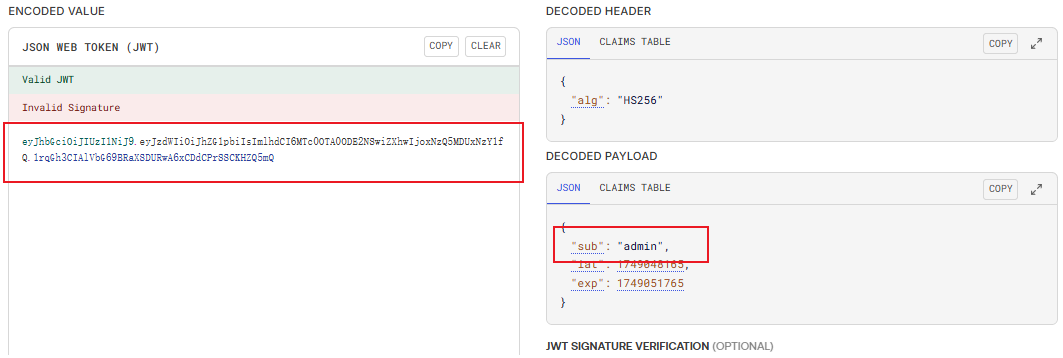
只要是固定的密钥，写在代码中，配置文件中，数据库中都一样，都算是硬编码

2、伪随机生成可爆破

采用随机数方法生成密钥，但是生成种子固定，导致可爆破的情况

2、jwt数据中的身份标识能否伪造

这个是什么意思呢，就比如说我们已经拿到密钥了，如果项目是使用用户名或者id值作为用户标识，进行后端校验，这个我们通常是有机会拿到的，例如这样



我们可以伪造用户名就可以登录，如果说它生成的时候使用的是用户名和密码呢？那么我们拿不到管理员密码也就是存在无法伪造的身份标识，那么就不存在危害，当然我这里说的是很单一的情况，实战环境中可能会有更多场景，这个要看项目具体代码实现和校验方式

关于JWT其实是可以延申很多点的，只要是项目采用可逆算法密钥泄露或者类似硬编码都会产生安全问题

实战案例

fastcms

```
  path: ./plugins

fastcms:
  auth:
    token:
      expire:
        seconds: 18000
      secret-key: SecretKey012345678901234567890123456789012345678901234567890123456789
  security:
    ignore:
      urls: "/fastcms/api/admin/login,\
```

nacos

```
nacos.core.auth.server.identity.key=serverIdentity
nacos.core.auth.server.identity.value=security

### worked when nacos.core.auth.system.type=nacos
### The token expiration in seconds:
nacos.core.auth.plugin.nacos.token.expire.seconds=18000
### The default token (Base64 String):
nacos.core.auth.plugin.nacos.token.secret.key=SecretKey012345678901234567890123456789012345678901234567890123456789

### worked when nacos.core.auth.system.type=ldapïXPLU{0} is Placeholder,replace login username
#nacos.core.auth.ldap.url=ldap://localhost:389
#nacos.core.auth.ldap.basedc=dc=example,dc=org
```